

LWFDSPC V0.18 變動摘要

2026-08-06 | commit `c8fa466` | tag `v0.18` | Modbus ID03 0x15 = `0x0018` | 已推送 `origin/main`

0 一句話總結

有動力倒溜偵測的計數方式由「連續」改為「淨」（反向 +1、正向 -1、地板 0），並在 EMB 上鎖 / 放鎖時清零計數器。前者讓門檻放大後仍能符合「倒溜不超過 1/4 車輪」規格，後者是前者的必要配套，否則會出現重新起步被立即重鎖的卡死。

標籤說明 `邏輯` = 會改變執行行為 `註解` = 僅文件 / 註解，不影響行為 `版本` = 版本號回報

類別	檔案	處數	內容
邏輯	main.c	3	淨計數（1 處）、LOCK / RELEASE 清零（2 處）
版本	s_modbus_decode.h	1	<code>FW_VER_MINOR</code> 6 → 8
註解	main.c / userparms.h s_logic_embraker.c/.h	6	語意由「連續」改為「淨」、修正過時的換算數字

合計 5 檔案、38 行新增 / 18 行刪除。行號會隨後續改動位移，用參數名或函式名搜尋最可靠。

1 程式邏輯變動 (會改變行為，共 2 項)

1-1. 反向邊緣計數：連續 → 淨

位置	舊	新	理由
main.c : 2852 CNRead_Inline()	<pre>} else { g_u8EmbRevEdgeCnt = 0; }</pre>	<pre>} else if (g_u8EmbRevEdgeCnt > 0u) { g_u8EmbRevEdgeCnt--; }</pre>	規格管的是淨倒溜位移，而連續計數只能界定單調位移。上坡與重力拉鋸時（倒溜→積分器充飽→推前一小段→再倒溜），連續計數每個循環都被歸零，淨位移卻早已超過 1/4 車輪 → 永遠不鎖。正向抵扣可讓拉鋸下的計數仍單調爬到門檻。

為什麼地板取 0 而不讓它變負：否則長距離正常前進會「預存額度」，後續真的倒溜時要先把負值填回來才會觸發，反而延誤偵測。

靜止抖動免疫仍在：抖動時方向交替 → 正負相抵停在 0 附近。

命令低於 `EMB_ROLLBACK_CMD_THRESHOLD` 的解除武裝分支維持直接歸零（未改）—— 那是「駕駛沒在下命令」，與拉鋸無關。

相容性：門檻小時（如原設計 $N=3$ ）本式與舊的連續計數行為幾乎不可區分，差異只在門檻放大後才顯現。

1-2. EMB 動作處清零計數器

位置	動作	新增	理由
main.c : 1432	ACTION_LOCK	<pre>g_u8EmbRevEdgeCnt = 0;</pre>	偵測已被執行 → 位移預算用掉。門鎖期間每個 tick 都會發 LOCK，故全程壓在 0

main.c : 1436	ACTION_RELEASE	<code>g_u8EmbRevEdgeCnt = 0;</code>	每次放煞車都從零起算 —— 打斷再鎖迴圈的關鍵。RELEASE 只在狀態轉移時發出 (平時為 ACTION_NONE) , 是一次性事件, 不會誤清正在累積的倒溜
---------------	----------------	-------------------------------------	---

不加這兩行會卡死。 `g_u8EmbRevEdgeCnt` 只在霍爾邊緣中斷內更新 (整段包在 `if (OldHallState != HallState)` 內) , 車被煞車夾住不動時整段完全不執行 —— 連「命令歸零 → 解除武裝歸零」的分支也不會跑 → 計數凍結在 $\geq$ 門檻。流程如下：

步	事件	EMB 狀態	計數	結果
1	倒溜達門檻 → LOCK + 門鎖	LOCKED	90	正常鎖定
2	鬆油門 → 解門	LOCKED	90	車不動 → 無邊緣 → 計數不變
3	重新給油門 → 運轉前檢查通過	RELEASED	90	放煞車
4	下一個 20 ms tick	→ LOCKED	90	立刻重鎖 (計數仍 ≥ 90) 。車一個邊緣都沒動就被鎖死, 而要扣掉計數又必須讓車動 → 卡死

這性質在改動前就存在, 但舊版靠運氣躲過: 夾煞車時車身小幅回彈只要出現一個正向邊緣就整個歸零。改成逐邊緣抵扣後, $N=90$ 需要 90 個正向邊緣才清得完, 回彈救不了 → 由「偶爾發生」變成「必然發生」。故這兩行是 1-1 的必要配套, 不是可選項。

寫入為單純覆寫 (非讀改寫) , 8-bit 在 16-bit MCU 上原子, 與 ISR 無競態。

2 行為對照 (舊 → 新)

情境	舊 (連續 + 同向歸零)	新 (淨 + 動作清零)	差異
單調重力倒溜	累加到 N 後鎖定	累加到 N 後鎖定	相同
PI 與重力拉鋸 (退 5 → 前 2 → 退 5...)	每循環歸零 → 可能永不觸發	淨值單調爬升 → 必然撞到 N	修正
恢復正常前進	一個邊緣即歸零	逐邊緣退回 0 (N 個邊緣內)	變慢
靜止抖動	歸零 (免疫)	正負相抵停在 0 附近 (免疫)	相同
鎖定後重新起步	靠回彈邊緣清零, 偶爾卡死	RELEASE 強制清零, 不會卡死	修正
門檻設小時 ($N=3$)	—	與舊版幾乎不可區分	相容

3 版本號變動

參數	位置	舊	新	說明
FW_VER_MINOR	s_modbus_decode.h : 16	6	8	打包 (CATEGORY<<8) (MAJOR<<4) MINOR → Modbus ID03 0x0015 回報 0x0018 · GUI 顯示 V0.18
(檔頭註解)	s_modbus_decode.h : 11~12	V0.16	V0.18	範例版本號同步更新

v0.17 是跳號的 (既有 tag 為 v0.13 / v0.15 / v0.16 , 本來就不連續) 。 **V0.x** 一律為內部測試版。

4 註解 / 文件修正 (不影響行為)

#	位置	修正內容
1	main.c : 241~244	<code>g_u8EmbRevEdgeCnt</code> 宣告註解改為「淨」計數，並加上 Δ 只在霍爾邊緣時更新 → 車靜止時凍結，故 LOCK/RELEASE 處必須清零
2	main.c : 1418~1425	新增 8 行說明：為何兩個分支都必須清零、卡死流程、原子性論證
3	main.c : 2830~2843	改寫計數區塊註解：淨計數定義、為何用淨而非連續、為何地板取 0、小門檻下的相容性
4	userparms.h : 279	修正過時換算： <code>EMB_ROLLBACK_CMD_THRESHOLD</code> 舊註解寫「183 馬達RPM = 0.35 km/h」（那是舊值 500 的換算），現值 100 應為 36.6 馬達RPM = 0.069 km/h ；比較符也由 <code>>=</code> 更正為 <code>></code> （程式用嚴格大於）
5	userparms.h : 269~273	段落說明改為淨計數，補上「規格管淨位移、拉鋸會使連續計數歸零」的理由
6	userparms.h : 280	<code>EMB_ROLLBACK_REV_EDGES</code> 註解「連續」→「淨」，補上 90 = 157 mm 的換算
7	s_logic_embraker.h : 48~50 s_logic_embraker.c : 132~133	偵測訊號描述由「連續 N 個反向邊緣」改為「淨反向邊緣數達 N（反向 +1、正向 -1、地板 0）」

5 驗證狀態與實車必測

已完成

項目	結果	方式
編譯 <code>main.c</code>	通過	XC16 v2.10，專案原設定 (<code>-mcpu=33CK256MP506 -O0 -Wall</code>)，無錯誤
編譯 <code>s_logic_embraker.c</code>	通過	同上
編譯 <code>s_modbus_decode.c</code>	通過	同上（版本號巨集）
實車行為	未測	見下方必測項目

僅做單檔編譯驗證，未做完整 build 與燒錄。

實車必測（順序有意義）

#	測項	判定
1	平地起停數次	停車後必須能重新起步，不可一給油門就被鎖。此項失敗 = 1-2 的清零沒生效，其餘測試無意義
2	坡上倒溜觸發 → 鬆油門 → 重新起步	直接驗證再鎖迴圈已消除。應能正常起步
3	坡上拉鋸	驗證淨計數確實會爬到門檻（舊版會被歸零）。建議用 X2CScope 同時看 <code>g_u8EmbRevEdgeCnt</code> 、 <code>piInputOmega.inReference</code> 、 <code>uGF.Direction</code>
4	Modbus 版本回報	ID03 位址 <code>0x0015</code> 應回 <code>0x0018</code> ，GUI 顯示 V0.18

6 本版未處理 (待實車數據後決定)

#	項目	現況	說明
1	EMB_ROLLBACK_REV_EDGES	90	≈ 157 mm 才鎖，貼著 1/4 車輪護欄上限 91 (159 mm)。淨計數生效後這顆值第一次會真的觸發，手感需實測。原設計值為 3 (5.2 mm)
2	調校文件同步	待改	quick_tuning_sheet_V0.16.html、 EMB_rollback_lock_guide_V0.16.html 內 REV_EDGES 仍寫 3、 CMD_THRESHOLD 仍寫 500，且尚未反映「連續→淨」的語意變更。建議等第 1 項定案後一次改

兩項均已寫入 commit message 與 tag 說明，不會遺失。